Semester should be Student/Education OS School staff falcuty and instution OS and and Bussniuss layer/OS idea is to centlize all eduction systems platioms and such into one super Eduction OS app

Student portfolio and public profile builder: showcase coursework, projects, and credentials with privacy controls
Computer
Campus business operations dashboard: map tuition, budgeting, and resource allocation across departments
Computer
Build the ER data model for the Education OS
Map out the Institution OS advisor workflow engine
Write the database schema for student profiles

Semester should be positioned as a unified **Education OS** that brings student experience, school operations, and business management into one connected platform. Current higher-ed systems increasingly emphasize centralized student records, connected advising workflows, and institution-wide integration across admissions, enrollment, finance, and student success. [1] [2] [3]

Operating model

The product can be structured as three visible layers on top of one shared data and workflow core. Modern higher-ed platforms increasingly connect students, faculty, staff, and administrators around one lifecycle record instead of isolated systems. [2] [4] [3]

Layer	Audience	Role
Student OS	Students, applicants, alumni	Profile, portfolio, registration, advising, messages, campus services
Institution OS	Faculty, advisors, registrar, admissions, student affairs, department staff	Student support workflows, academic operations, service delivery
Business OS	Finance, leadership, planning, operations	Tuition, budgeting, resource allocation, performance, forecasting
Education OS Core	Entire institution	Shared identity, workflow engine, integrations, analytics, governance

Education OS ER model

The ER model should unify person identity, academic records, advising, finance, organizational structure, and communication systems. Centralized SIS and institutional-impact platforms are built around one connected record model spanning admissions, enrollment, academic progress, financial processes, and support actions. [5] [3] [2]

Core entities

Entity	Purpose
`persons`	Canonical identity across prospect, applicant, student, faculty, staff, alumni
`user_accounts`	Authentication and access control
`person_roles`	Role assignment by context, term, or organization
`profiles`	Public/private profile and portfolio identity
`organizations`	Colleges, departments, offices, clubs, programs
`programs`	Degrees, majors, certificates, pathways
`terms`	Academic periods
`course_catalog`	Master course definitions
`course_sections`	Scheduled offerings by term
`enrollments`	Section registrations and outcomes
`applications`	Admissions applications and lifecycle state
`advising_cases`	Success cases, interventions, referrals
`degree_plans`	Student academic plans by term
`appointments`	Advising and service appointments
`holds`	Registration, finance, compliance, advising holds
`charges`	Tuition, fees, other student-account charges
`aid_awards`	Scholarships, aid, grants
`budgets`	Departmental and institutional budget plans
`resource_allocations`	Budget distribution by unit, initiative, or term
`messages`	Messaging records
`notifications`	Event-driven alerts and announcements

High-level relationships

```
persons
  ├─ user_accounts
  ├─ person_roles
  ├─ profiles
  ├─ applications
  ├─ students
  │   ├─ enrollments
  │   ├─ degree_plans
  │   ├─ holds
  │   ├─ advising_cases
  │   ├─ appointments
  │   ├─ charges
  │   └─ aid_awards
  └─ staff_assignments

organizations
  ├─ programs
  ├─ course_catalog
  ├─ budgets
  └─ resource_allocations

terms
  ├─ course_sections
  ├─ enrollments
  ├─ applications
  └─ budgets
```

Student profile schema

Student profiles should support identity, portfolio, credentials, and granular privacy rules. Student-success and CRM platforms increasingly emphasize complete student profiles, shared documents, progress context, and mobile self-service as part of the student record. [6] [1]

Profile capabilities

- Public professional profile.
- Campus-only identity profile.
- Private student record extensions.
- Portfolio entries for coursework, projects, credentials, and media.
- Field-level privacy and share settings.
- Verified institution-issued achievements.

Database schema

```
CREATE TABLE persons (
  id uuid PRIMARY KEY,
  institutional_id text UNIQUE,
  lifecycle_stage text NOT NULL CHECK (
    lifecycle_stage IN ('prospect','applicant','admitted','matriculating','student',
  ),
  legal_first_name text,
  legal_last_name text,
  preferred_name text,
  primary_email text UNIQUE NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE profiles (
  id uuid PRIMARY KEY,
  person_id uuid NOT NULL UNIQUE REFERENCES persons(id),
  profile_slug text UNIQUE,
  headline text,
  bio text,
  photo_url text,
  major_label text,
  class_year text,
  hometown text,
  pronouns text,
  default_visibility text NOT NULL CHECK (
    default_visibility IN ('public','campus','connections','private')
  ),
  messaging_enabled boolean NOT NULL DEFAULT true,
  portfolio_enabled boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE profile_fields (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  field_key text NOT NULL,
  field_value jsonb NOT NULL,
  UNIQUE(profile_id, field_key)
);

CREATE TABLE profile_visibility_rules (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  field_key text NOT NULL,
  audience_type text NOT NULL CHECK (
    audience_type IN ('public','campus','connections','advisor','faculty','org','priv
  ),
  audience_value text,
  can_view boolean NOT NULL DEFAULT true
);

CREATE TABLE portfolio_items (
  id uuid PRIMARY KEY,
```

```
  profile_id uuid NOT NULL REFERENCES profiles(id),
  item_type text NOT NULL CHECK (
    item_type IN ('coursework','project','credential','award','publication','presenta
  ),
  title text NOT NULL,
  subtitle text,
  description text,
  evidence_url text,
  media_url text,
  issued_by_org_id uuid,
  related_course_id uuid,
  visibility text NOT NULL CHECK (
    visibility IN ('public','campus','connections','private')
  ),
  verification_status text NOT NULL DEFAULT 'self_added' CHECK (
    verification_status IN ('self_added','faculty_verified','institution_verified')
  ),
  start_date date,
  end_date date,
  sort_order integer NOT NULL DEFAULT 0
);

CREATE TABLE credentials (
  id uuid PRIMARY KEY,
  person_id uuid NOT NULL REFERENCES persons(id),
  credential_type text NOT NULL CHECK (
    credential_type IN ('badge','certificate','microcredential','degree_progress_mile
  ),
  title text NOT NULL,
  issuer_org_id uuid,
  issued_at timestamptz,
  expires_at timestamptz,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb
);

CREATE TABLE profile_links (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  label text NOT NULL,
  url text NOT NULL,
  visibility text NOT NULL CHECK (
    visibility IN ('public','campus','private')
  )
);
```

Student portfolio and public profile builder

The Student OS should let students present verified academic and co-curricular work with privacy controls. Advising and student-success systems increasingly pair student-facing mobile experiences with documents, to-dos, planning, and profile context. [3] [1]

Builder modules

Module	Function
Identity card	Photo, headline, major, class year, interests
Academic showcase	Coursework, capstones, labs, writing, grades optional by policy
Project gallery	Design, code, research, presentations, media
Credential wallet	Certificates, badges, honors, milestones
Experience log	Leadership, internships, campus jobs, service
Privacy center	Per-item and per-field visibility controls
Share tools	Public URL, advisor share, campus share, employer share

Posting model

```
Create entry
  → Choose content type
  → Add title, description, media, links
  → Link to course, club, credential, or experience
  → Select visibility
  → Optional faculty or institution verification
  → Publish to profile and eligible feeds
```

Institution OS advisor workflow engine

Advisor workflow should operate as a coordinated-care engine, not just scheduling software. Current advising and student-success systems stress shared notes, referrals, early alerts, appointments, to-dos, messaging, and coordinated care networks. [7] [8] [1]

Core workflow entities

Entity	Purpose
`advisor_assignments`	Maps students to advisors, coaches, faculty mentors
`advising_cases`	Intervention, success, or support case container
`advising_alerts`	Risk flags from faculty, LMS, registration, finance, attendance
`advising_tasks`	Follow-ups, student actions, staff actions
`advising_referrals`	Routed support to tutoring, aid, counseling, registrar
`advising_notes`	Shared or restricted documentation
`appointments`	Scheduled advising interactions
`journeys`	Milestone-based student workflows by stage

Workflow engine

```
Signal received
  → Create alert or task
  → Match student to assigned advisor/care team
  → Evaluate severity and routing rules
  → Open advising case or attach to existing case
  → Notify advisor and student
  → Schedule appointment / outreach / referral
  → Track actions and outcomes
  → Close case or escalate
```

Rule examples

- Missed registration window creates outreach task for assigned advisor.
- Low course engagement creates early alert and optional faculty progress report review.
- Financial hold creates a joint advising and student-finance referral.
- Admitted student without orientation completion creates transition case before matriculation.

Staff views

View	Purpose
Caseload dashboard	Prioritize students by urgency, stage, and risk
Student 360	Academic, financial, engagement, and communication context
Appointment board	Upcoming appointments, no-shows, drop-ins
Referral tracker	Open handoffs across offices
Journey monitor	Which students are missing milestone steps
Outcome analytics	Intervention results and unresolved bottlenecks

Registration, tuition, budgeting, and resource allocation

The Business OS should connect tuition, budget planning, and resource allocation across departments into one management layer. Institutions increasingly use dashboards that compare budget versus actuals, compensation planning, and resource commitment allocation to support campus business officers. [9] [10]

Business operations dashboard

Area	Key metrics
Tuition operations	Net tuition, receivables, payment plan adoption, discount rate
Budget performance	Budget vs actual, YTD variance, carryforward, forecast
Department planning	Spend by college, department, program, initiative

Area	Key metrics
Resource allocation	Strategic-priority funding, staffing, facilities, technology
Compensation planning	Payroll vs budget, open positions, labor allocation
Academic economics	Section demand, instructor load, seat utilization
Student success ROI	Advising staffing, intervention volume, persistence outcomes

Core finance entities

```
organizations
  ├─ budgets
  ├─ budget_lines
  ├─ actuals
  ├─ resource_allocations
  ├─ position_plans
  └─ cost_centers

students
  ├─ charges
  ├─ payments
  ├─ aid_awards
  └─ account_balances
```

SCUP's resource-allocation framework emphasizes aligning assessment, strategy, budgeting, and departmental requests so funding decisions reflect institutional priorities rather than disconnected requests. [10]

Education OS database domains

A practical Education OS schema should be divided into major bounded domains.

Domain map

Domain	Tables
Identity	`persons`, `user_accounts`, `roles`, `permissions`, `sessions`
Profile and portfolio	`profiles`, `profile_fields`, `profile_visibility_rules`, `portfolio_items`, `credentials`
Admissions	`applications`, `application_documents`, `application_reviews`, `decisions`
Academics	`programs`, `terms`, `course_catalog`, `course_sections`, `enrollments`, `degree_plans`
Advising	`advisor_assignments`, `advising_cases`, `advising_alerts`, `advising_notes`, `referrals`, `appointments`
Finance and business	`charges`, `payments`, `aid_awards`, `budgets`, `budget_lines`, `resource_allocations`
Communication	`conversations`, `messages`, `announcements`, `notifications`

Domain	Tables
Governance	`audit_events`, `retention_rules`, `consents`, `policy_exceptions`

Recommended product framing

Semester should be described as:

- **Student OS** for profiles, planning, portfolio, registration, messaging, and support.
- **Institution OS** for faculty, advising, registrar, admissions, and coordinated care.
- **Business OS** for finance, planning, budgeting, and resource allocation.
- **Education OS** as the shared data, workflow, identity, and analytics platform.

That framing aligns with how SIS, advising CRM, and institutional intelligence platforms describe centralized student systems, coordinated care, and institution-wide operational models. [1] [2] [3]

Would you like the next version as a full technical architecture document with APIs, services, workflows, and a complete SQL schema across all four layers?



1. https://eab.com/solutions/navigate360/
2. https://www.ellucian.com/blog/what-student-information-system-higher-ed
3. https://civitaslearning.com/
4. https://listedtech.com/student-information-systems/
5. https://www.higher-education-marketing.com/student-portal/student-information-system
6. https://eab.com/resources/video/navigate360-student-crm-for-community-colleges/
7. https://e2eadvising.com/
8. https://www.ellucian.com/products/student/student-success/advising
9. https://blink.ucsd.edu/finance/bi-financial-reporting/plan-bud-dashboard/index.html
10. https://www.scup.org/resource/a-guide-for-optimizing-resource-allocation/
11. https://www.gartner.com/reviews/market/higher-education-student-success-platforms